

Obstáculos para una llama

Una llama quiere viajar a través de la Meseta Andina. La llama tiene un mapa de la meseta en forma de grilla con $N \times M$ celdas cuadradas. Las filas del mapa están enumeradas de 0 a $N - 1$ de arriba hacia abajo, y las columnas están enumeradas de 0 a $M - 1$ de izquierda a derecha. Una celda en la fila i y la columna j ($0 \leq i < N, 0 \leq j < M$) esta denotada por (i, j) .

La llama ha estudiado el clima de la meseta y descubrió que todas las celdas de una misma fila del mapa tienen la misma **temperatura** y todas las celdas de una misma columna del mapa tienen la misma **humedad**. La llama te dará dos arreglos de números enteros T y H de tamaño N y M respectivamente. Aquí $T[i]$ ($0 \leq i < N$) indica la temperatura de todas las celdas en la fila i , y $H[j]$ ($0 \leq j < M$) indica la humedad de todas las celdas en la columna j .

La llama también ha estudiado la flora de la meseta y llego a notar que una celda (i, j) esta **libre de vegetación** si y solo si su temperatura es mayor a su humedad, más formalmente si $T[i] > H[j]$.

La llama puede viajar a través de la meseta solo recorriendo **caminos validos**. Un camino valido es una secuencia de celdas distintas que satisfacen las siguientes condiciones:

- Cada par de celdas consecutivas en el camino tienen un lado en común.
- Todas las celdas del camino están libres de vegetación.

Tu tarea consiste en responder Q preguntas. Para cada pregunta, se te dan cuatro números enteros: L, R, S , y D . Debes determinar si es que existe un camino valido tal que:

- El camino inicie en la celda $(0, S)$ y termine en la celda $(0, D)$.
- Todas las celdas en el camino están entre las columnas L y R , inclusive.

Se garantiza que ambas $(0, S)$ y $(0, D)$ están libres de vegetación.

Detalles de implementación

La primera función que debes implementar es:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : un arreglo de tamaño N especificando la temperatura en cada fila.
- H : un arreglo de tamaño M especificando la humedad en cada columna.

- Esta función es llamada exactamente una vez por cada caso de prueba, antes de cualquier llamada a la función `can_reach`.

La segunda función que debes implementar es:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : los enteros describiendo una pregunta.
- Esta función es llamada exactamente Q veces por cada caso de prueba.

Esta función debe retornar `true` sí y solo si existe un camino valido desde la celda $(0, S)$ a la celda $(0, D)$, tal que todas las celdas en el camino estén entre las columnas L y R , inclusive.

Restricciones

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ para todo i tal que $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ para todo j tal que $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Ambas celdas $(0, S)$ y $(0, D)$ están libres de vegetación.

Subtareas

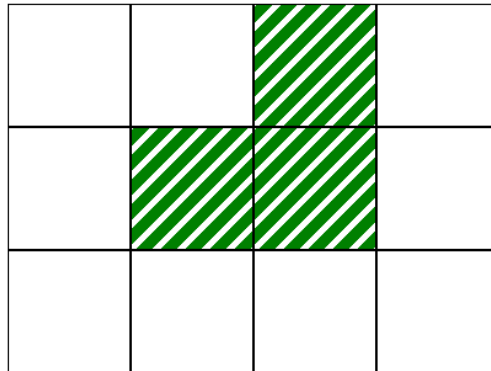
Subtarea	Puntaje	Restricciones adicionales
1	10	$L = 0, R = M - 1$ para toda pregunta. $N = 1$.
2	14	$L = 0, R = M - 1$ para toda pregunta. $T[i - 1] \leq T[i]$ para todo i tal que $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ para toda pregunta. $N = 3$ y $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ para toda pregunta. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ para toda pregunta.
6	17	Sin restricciones adicionales.

Ejemplo

Considera la siguiente llamada a la función:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

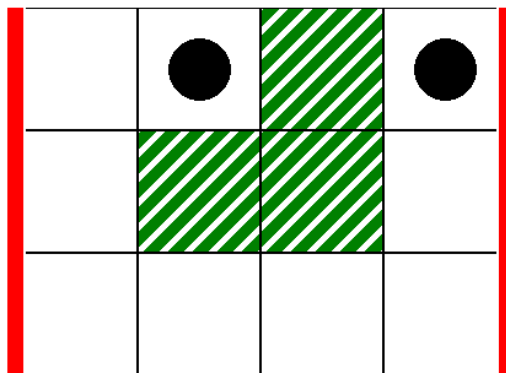
Esto corresponde al mapa en la siguiente imagen, donde las celdas de color blanco están libres de vegetación:



Como primera pregunta, considera la siguiente llamada a la función:

```
can_reach(0, 3, 1, 3)
```

Esto corresponde al escenario en la siguiente imagen, donde las líneas rojas indican el rango de columnas de $L = 0$ a $R = 3$, y los puntos negros indican las celdas inicial y final:



En este caso, la llama puede llegar desde la celda $(0,1)$ a la celda $(0,3)$ a través del siguiente camino valido:

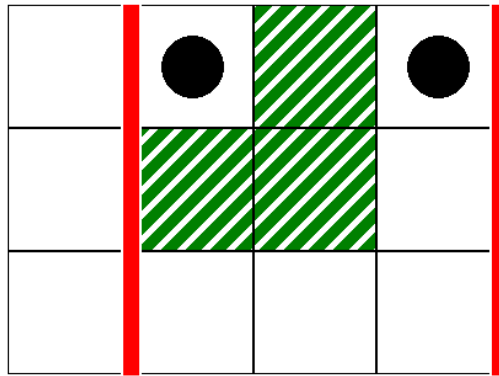
$(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)$

Por lo tanto, está llamada a la función debe retornar `true`.

Como segunda pregunta, considera la siguiente llamada a la función:

```
can_reach(1, 3, 1, 3)
```

Esto corresponde al escenario en la siguiente imagen:



En este caso, no existe un camino valido desde la celda $(0,1)$ a la celda $(0,3)$, tal que todas las celdas en el camino estén entre las columnas 1 a 3, inclusive. Por lo tanto, está llamada a la función debe retornar `false`.

Evaluador

Formato de Entrada:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Aquí, $L[k]$, $R[k]$, $S[k]$ y $D[k]$ ($0 \leq k < Q$) especifican los parámetros para cada llamada a la función `can_reach`.

Formato de Salida:

```
A[0]
A[1]
...
A[Q-1]
```

Aquí, $A[k]$ ($0 \leq k < Q$) es 1 si es que la llamada a la función `can_reach(L[k], R[k], S[k], D[k])` retorno `true`, y 0 en otro caso.